

# 教學文件

程式碼註解裡有完整的公式推導，這裡放的是**程式碼講不了的東西**：流程圖、全景視角、以及做簡報時的敘事順序。

## 建議順序

|   | 文件                      | 對應程式碼                                        | 一句話                                      |
|---|-------------------------|----------------------------------------------|------------------------------------------|
| 1 | 全景圖                     | —                                            | 一句話進去，到權重被改動，中間發生了什麼                     |
| 2 | 詞表大小是怎麼決定的              | <code>tokenizer/bpe.py</code>                | 「6400 個詞哪來的」                             |
| 3 | Embedding 與 scatter-add | <code>scratch/embedding.py</code>            | 查表為什麼是矩陣乘法，反向為什麼是相加                      |
| 4 | 反向傳播全鏈                  | 全部                                           | 從 <code>p - y</code> 一路走回 <code>E</code> |
| 5 | 權重是怎麼被改的                | <code>scratch/adamw.py</code>                | 反向算完之後才輪到 optimizer                      |
| 6 | 三個最常見的誤解                | —                                            | 做簡報時最值得講的三件事                             |
| 7 | 權重與 tokenizer 綁定        | <code>tests/test_tokenizer_binding.py</code> | 為什麼 continual pre-training 必須用同一份詞表      |

## 附：怎麼產生講解用的素材

```
# 逐層梯度驗證表（可直接貼進簡報）
python tests/test_layers.py          # -> traces/gradcheck.txt

# BPE 合併過程，看詞表怎麼從 3653 長到 6400
python tests/test_bpe.py             # -> traces/bpe_training.txt

# 第 1 步的完整數值軌跡（前向 + 反向 + AdamW 每個中間量）
python train.py --trace-steps 1      # -> traces/train_step_0001.txt

# 一次推論的完整前向
python generate.py --prompt "貓是一種" --trace # -> traces/inference.txt

# 證明權重與 tokenizer 綁死（用錯會比沒訓練還糟）
python tests/test_tokenizer_binding.py
```

`traces/sample/` 裡有一份預先跑好的，不用執行就能看。